5.5.5 Roll back (Abort)

Undo modification of database file which are done by failure transaction.

5.6 Durability

- Durability maintained by Recovery management component of DBMS Software.
- The transaction should able to recover under any case of failure.
- Transaction fails because of

1. Power failure
2. Software crash
   - OS restarted
   - DBMS restarted
3. OS/DBMS concurrency controller may kill transaction.
4. Hardware Crash (Disk failure)
   RAID architecture of disk design is used to overcome the problem.

5.7 Consistency

- User responsible for consistency.
- Database should be consistent before and after the execution of the transaction.
- DB operations requested by user (SQL queries/transaction operation) must be logically correct.

5.8 Isolation

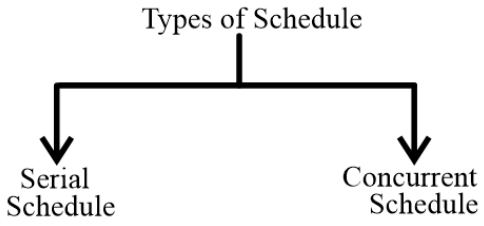
- Isolation maintained by concurrency control component.
- Isolation means concurrent execution of 2 or more transaction result must be equal to result of some serial schedule.

5.9 Schedules

Time order execution sequence of two or more transactions.

Example :

S : $R_2(A)$ $R_1(A)$ $W_3(A)$



5.9.1 Serial Schedule

- After Commit of one transaction, begins (Start) another transaction.
- Number of possible serial Schedules with 'n' transactions is "n!"
- The execution sequence of Serial Schedule always generates consistent result.

Example

S : $R_1(A)$ $W_1(A)$ Commit (T_1) $R_2(A)W_2(A)$ commit (T_2).

5.9.2 Advantage

- Serial Schedule always produce correct result (integrity guaranteed) as no resource sharing.

5.9.3 Disadvantage

- Less degree of concurrency.
- Through put of system is low.
- It allows transactions to execute one after another.

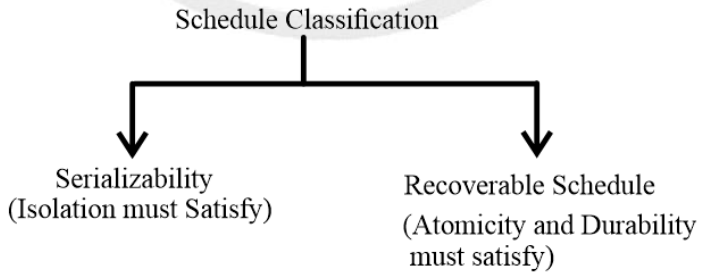
5.10 Concurrent Schedule

Transactions can execute concurrently or simultaneously

- May result inconsistency.
- Better through put and less response time.
- To maintain consistency transaction should satisfy ACID property.
- If T_1 and T_2 transaction with 'n' and 'm' operations each, then

  No. of concurrent Schedule = $\frac{(n+m)!}{n!\ m!}$

5.11 Schedule Classification

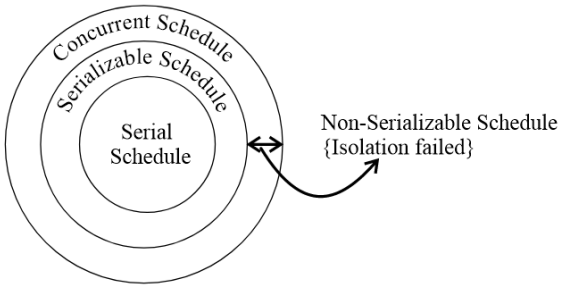


5.12 Serializable Schedule

A Schedule is serializable Schedule if it is equivalent to a Serial Schedule.

(i) Conflict Serializability

(ii) View Serializability



5.13 Conflict Serializability

5.13.1 Topological order

Graph traversal algorithm for directed graph

(i) Visit vertex (v), whose indegree is 'O' and delete 'V' from the graph.

(ii) Repeate (i) for all the vertices of graph.

5.13.2 Conflict Pairs

Two operations form Schedule (s) are conflict pair if and only if

(i) Atleast one write operation.

(ii) Operation on different data item.

(iii) Operations are from different transactions.

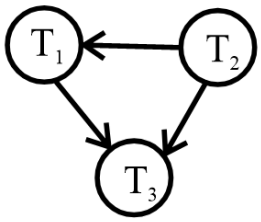
$$\left.\begin{array}{l} S: r_i(A).....w_j(A) \\ S: w_i(A).....r_j(A) \\ S: w_i(A).....w_j(A) \end{array}\right\} \text{Conflict Pairs}$$

5.13.3 Precedence Graph

A graph G in which vertex (v) represent the transaction of Schedule and edges (E) represent conflict pair precedence's.

Example:

S : $R_2(A)$ $R_2(B)$ $W_1(A)$ $W_2(B)$ $W_3(A)$ $W_3(B)$



5.13.4 Conflict Equal Schedule

Schedule S_1 and S_2 are conflict equal Schedule if and only if Schedule S_2 can derived by inter changing consecutive non-conflict pairs of Schedule S_1.

$$S_1 \xleftarrow[\text{Consecutive non-conflict pair}]{\text{Interchange}} S_2$$

Note:

If Schedule S_1 and S_2 have

(i) Same set of transactions, and

(ii) Same precedence graph and

(iii) A cyclic precedence graph then

S_1 and S_2 are conflict equal Schedule.

Important Point 1:

1. If S_1, S_2 Schedule are conflict equal then precedence graph of S_1 and S_2 must be same.
2. If S_1 and S_2 have same precedence graph then S_1 and S_2 may or may not conflict equal.

5.14 Conflict Serializable Schedule

Schedule (s) is conflict serializable Schedule if and only if some serial Schedule (S) must be conflict equal to Schedule (S).

$$\underbrace{\text{Schedule (s)}}_{\text{Conflict Serializable Scchedule}} \xleftrightarrow[\text{Equal}]{\text{Conflict}} (S') \text{Serial Schedule}$$

Important Point 2:

Schedule (s) is conflict serializable Schedule if and only if

1. Precedence graph is a cyclic and
2. Conflict equal serial schedule are to apological order of precedence graph.

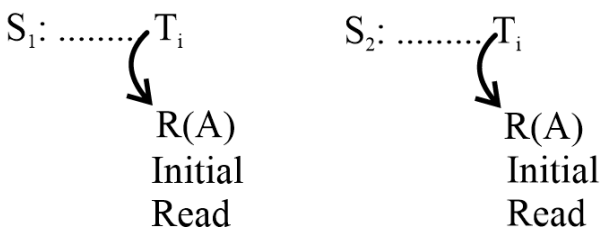
Note:

[No. of serial schedule conflict equal to schedule s] = [No. of topological order of schedules precedence graph]

5.15 View Serializable Schedule

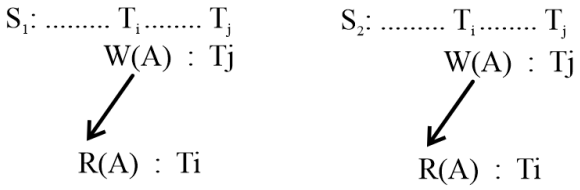
- Schedule (s) is view serializable if and only if some serial schedule (s') view equal to schedule (s).
- Let S_1 and S_2 be two schedule with the same set of transactions, S_1 and S_2 are view equal view equivalent if the following three conditions are met:

5.15.1 Initial Read



For each data item A, if transaction T_i reads the initial value of A in Schedule S_1, then transaction T_i must, in Schedule S_2, also read the initial value of A.

5.15.2 Updated Read



For each data item A if transaction T_i perform Read (A) in Schedule S_1 and that value was produced by transaction T_j, then transaction Ti must in Schedule S_2 also read the value of A that is produce by the transaction Tj.

5.15.3 Final Write

S_1 : ..... Ti .....

Final write of A : $w(A)$ (Last write)

S_2 : ..... Ti....

Final write of A : $w(A)$ (Last write)

For each data item A, the transaction that perform the final write (A) operation in schedule (S_1) must perform the final write (A) operation in schedule S_2.

Important Point 3

- If schedule S is conflict serializable schedule, then S is also view serializable schedule.
- If schedule s is not conflict serializable then it may or may not view serializable
- Every view serializable schedule that is not conflict serializable.

5.16 Classification based on Recoverability

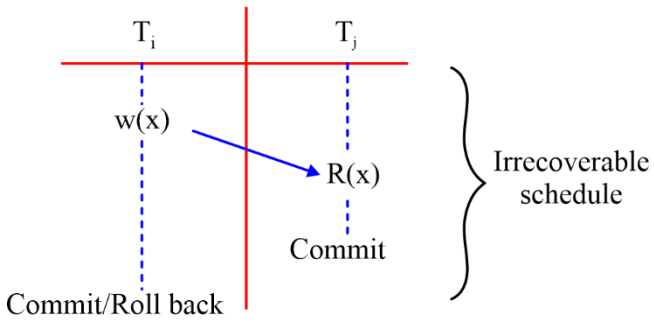
The concurrent execution may leads:

1. Irrecoverable schedule
2. Cascading rollback problem
3. Lost update problem

The above problems can occur even if the schedule is serializable (Integrity satisfied)

5.16.1 Irrecoverable Schedule

- A schedule(s) is irrecoverable if and only if transaction T_j reads data item 'x', which is updated by T_i and commit of T_j before commit/rollback of transaction T_i.
- Irrecoverable means unable to recover or rollback.



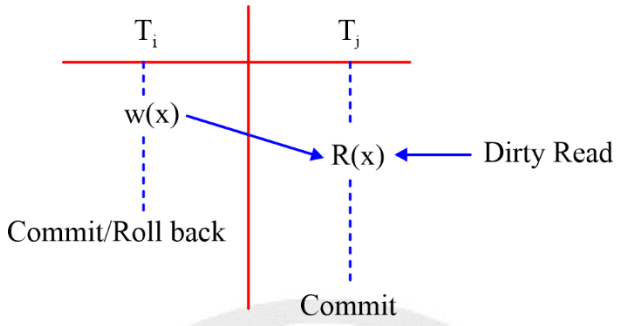
5.16.2 Recoverable Schedule

A schedule(s) is recoverable if and only if

(I) No uncommitted read (no dirty ready in schedule(s).

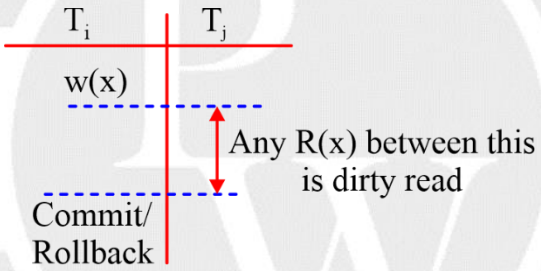
Or

(II) If transaction T_j reads data item 'x' which is updated by transaction 'T_i', then commit of T_j must be delayed until commit/rollback of T_i.



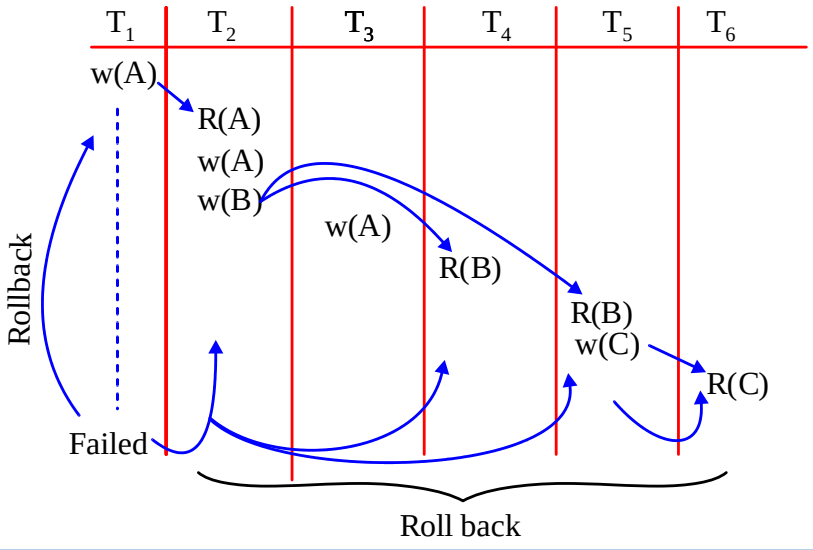
5.16.3 Uncommitted Read (Dirty Read)

Transaction T_j reads data item 'x' which is updated by uncommitted transaction. T_i



5.17 Cascading Rollback Schedule (Problem)

When some transaction reads data items which is updated by some other transaction then because of failure of the transaction that updated the data item may rollback all the dependent transaction.

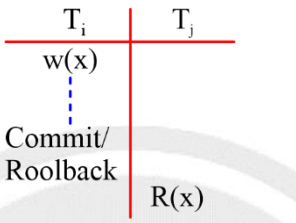


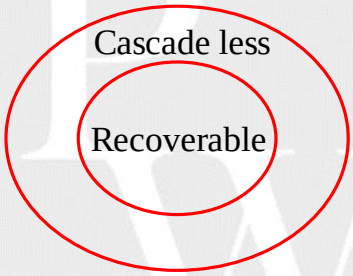
Failure of T_1 forced to rollback T_2, T_4, T_5 and T_6.

- **Disadvantage of Cascading Rollback**
  1. It waste the CPU execution time.
  2. Wastage of I/O access cost.

5.17.1 Cascadeless Rollback Recoverable Schedule

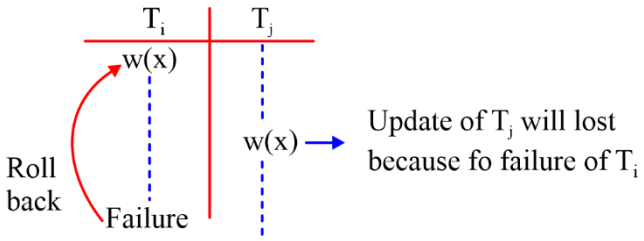
- No dirty read in the schedule.
- Transaction T_j should delay read (x) which is updated by T_i, until T_i commit or rollback.





5.18 Lost Update Problem

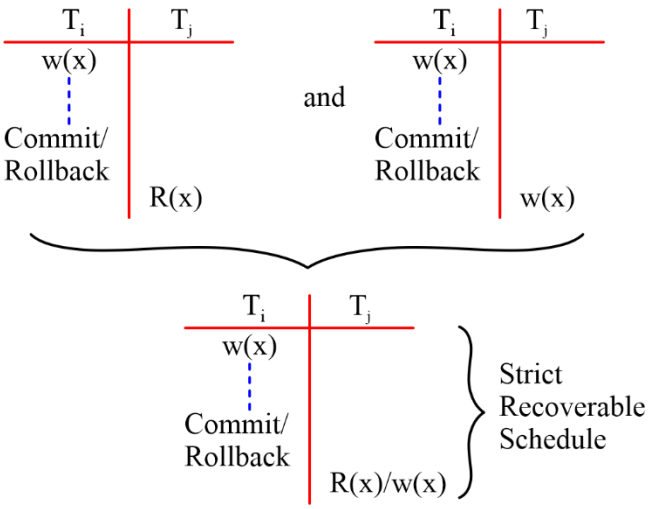
Lost update problem occur if T_j write (x) which is already written by uncommitted transaction T_i.

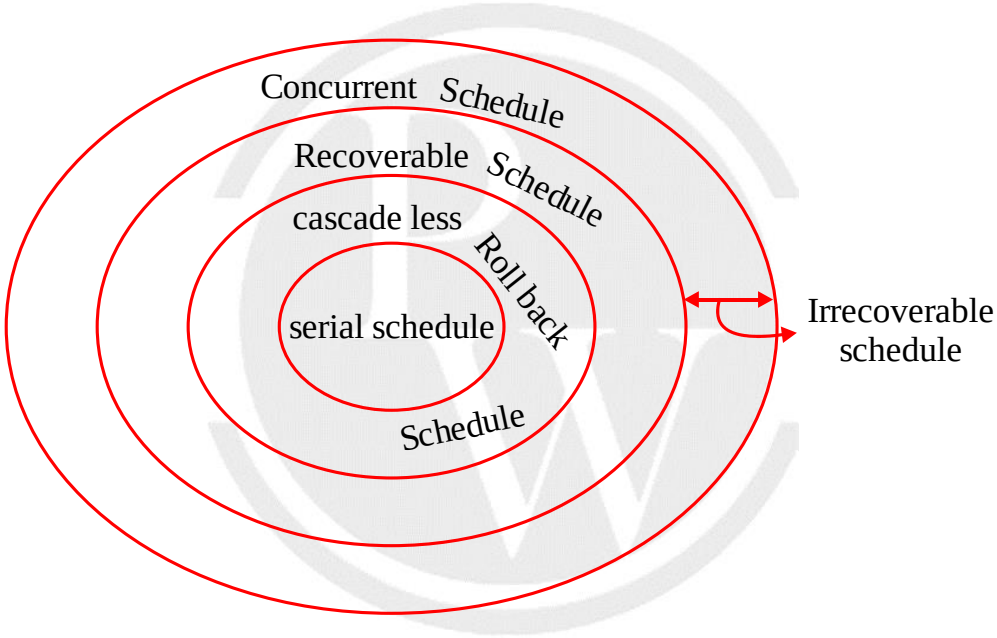


5.19 Strict Recoverable schedule

A schedule must satisfy

1. Cascadeless rollaback recoverable schedule and
2. If T_i writes (x) then other transaction T_j write (x) must be delayed until T_i commit or rollback.





Note:

1. Strict recoverable schedule may or may not be serializable.
2. Serializable schedule may or may not be recoverable.